SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Concept Mapping

Course Code:	CSA0309	**Course Title:**	Data Structures
CO Assessed:	CO1 – Imitation (BL3, BL4)	**Assessment:**	Assessment Tool 2 – Concept Mapping
Weightage:	35% of CIA	**Total Marks:**	100
CO1 Statement:	*Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity.*		
Student Name:	Y.SAI GOPAL REDDY	**Reg. No.:**	19252483
Section / Batch:		**Date:**	08/07/2026

Code of Conduct

I Y.SAIGOPALREDDY(192524083)________________________________(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: SAIGOPALREDDY____

Instructions

- This test contains 5 Concept mapping questions. Total: 100 Marks.
- When a student answer using a concept map, in evaluation the answer should be with following four requirements: Understanding of concepts, Connections between ideas, Logical structure, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.

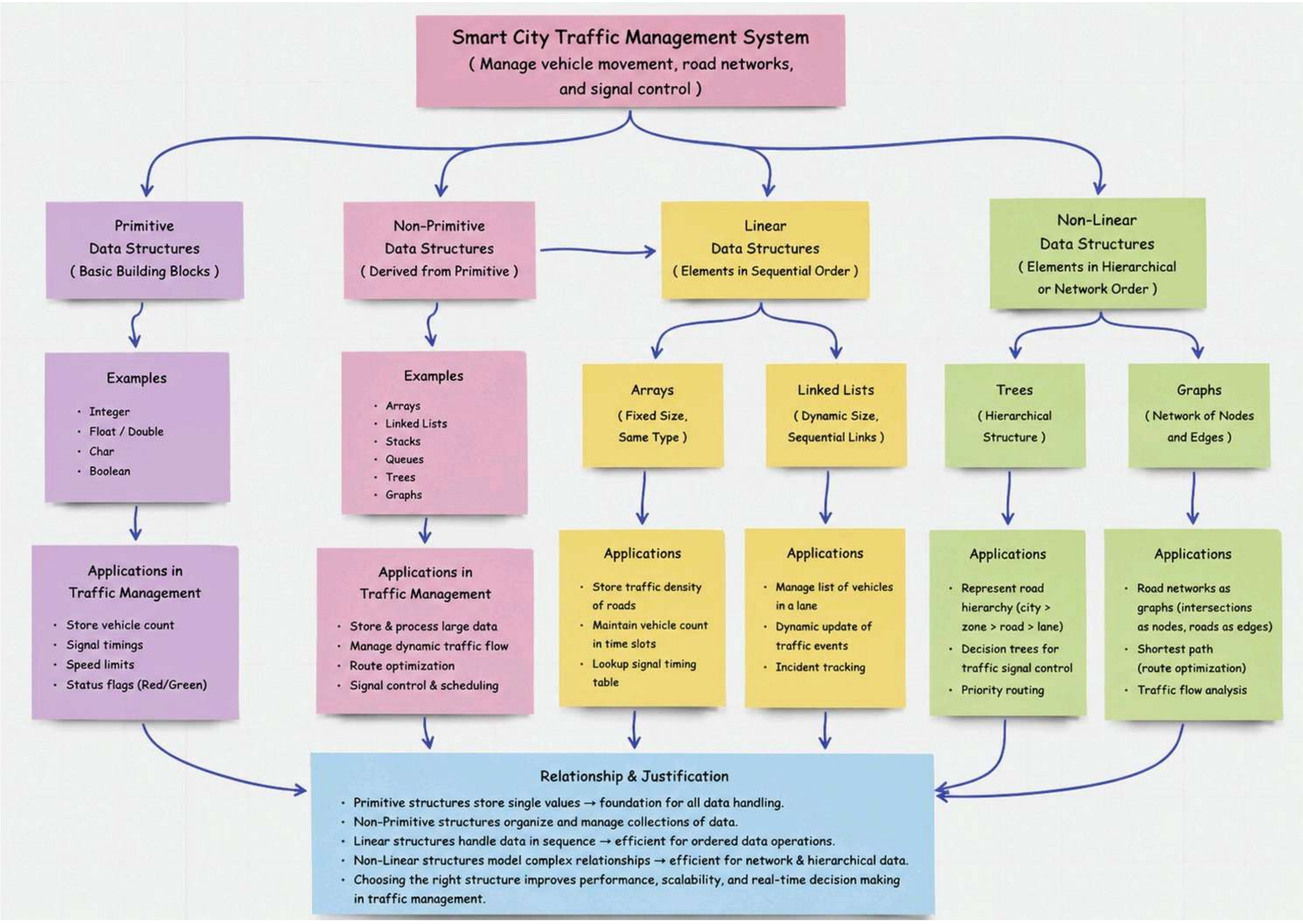
Section / Topic	Focus	Q Nos	Marks
Linked data structure	Basic data structure	1	20
Primitive and Non-Primitive Data Structure - Linear and Non-Linear Data Structure	Classifications of Linear data structure	2	20
Search Data Structure	Linear Search and Binary Search	3	20
Recursion, Performance analysis	Recursion	4	20
Tree Data Structures	Classifications of Tree data structure	5	20
	GRAND TOTAL:	**100 Marks**	

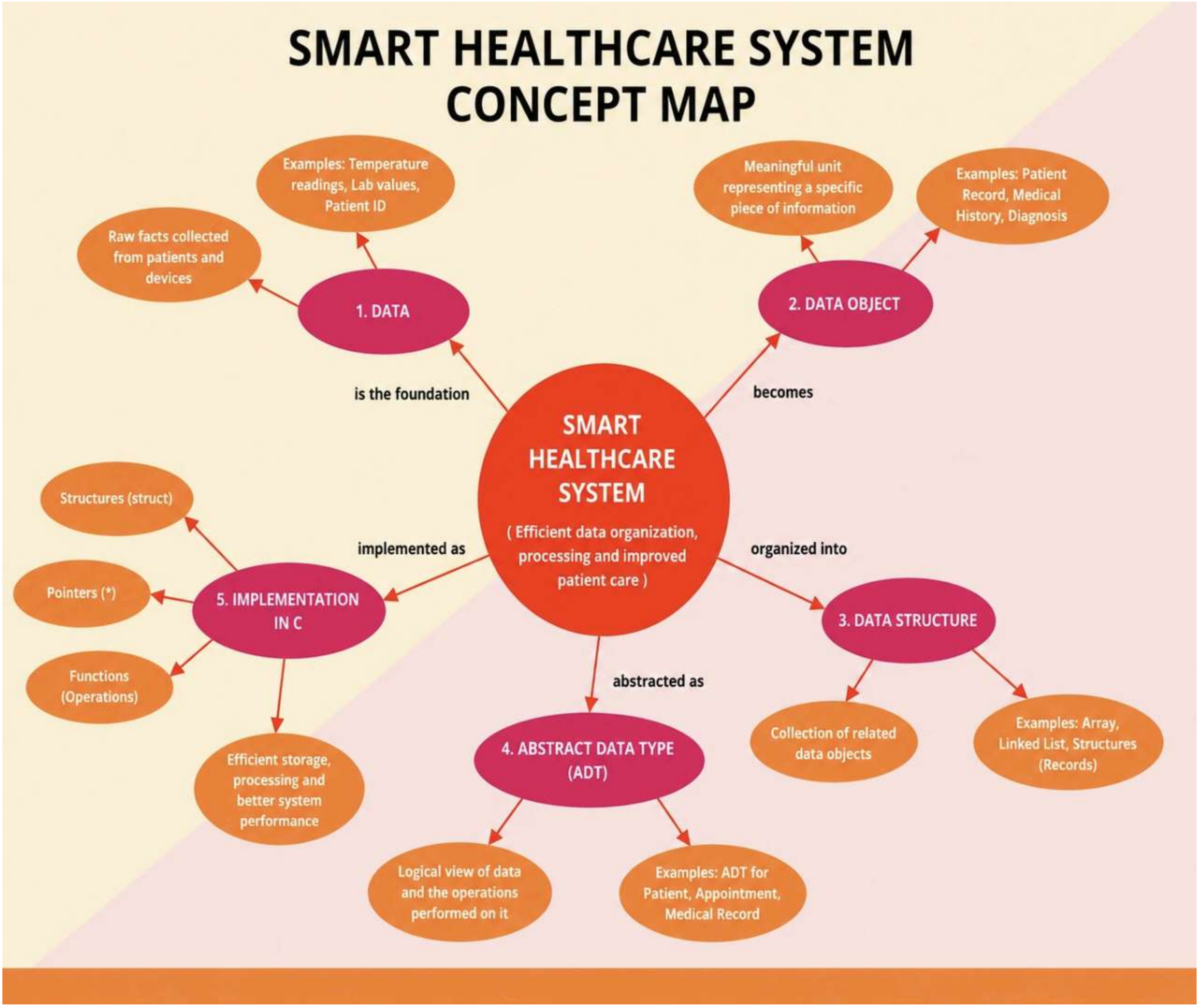
Annexure A – Concept Mapping

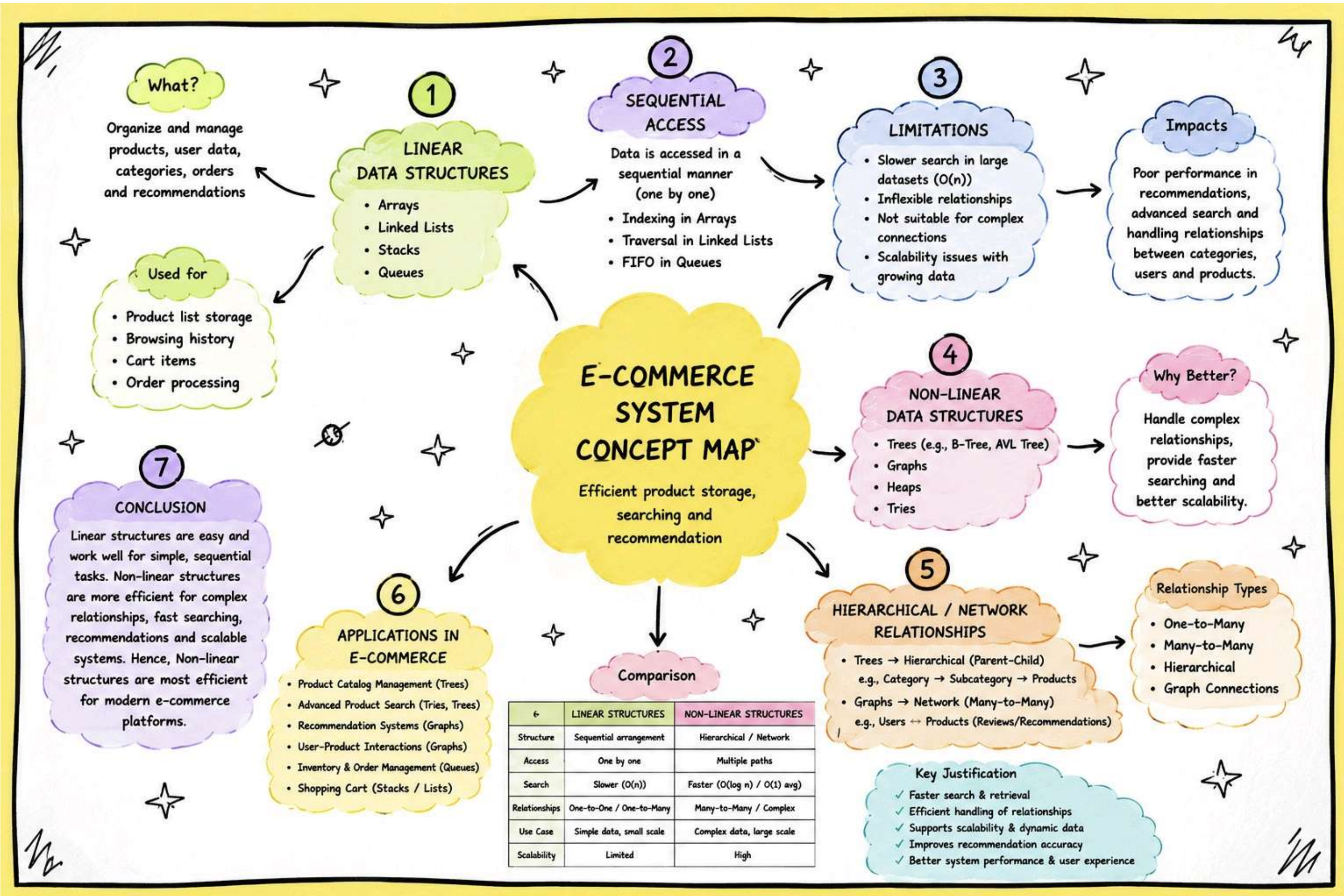
1. A smart city traffic management system handles data related to vehicle movement, road networks, and signal control. Construct a concept map connecting Primitive Data Structures → Non-Primitive Data Structures → Linear → Non-Linear Structures. Include examples such as arrays, linked lists, trees, and graphs, and illustrate their applications in traffic management. Explain the relationships between these structures and justify their classification based on functionality and efficiency. (20 Marks)
2. A smart healthcare system processes large volumes of patient data such as temperature readings, medical history, and diagnostic details. Construct a concept map linking the progression from Data → Data Object → Data Structure → Abstract Data Type (ADT) → Implementation in C. Identify relevant sub-concepts such as structures, pointers, and operations, and establish meaningful relationships among them. Organize the concept map hierarchically and justify how each level contributes to efficient data organization, processing, and improved system performance. (20 Marks)
3. E-commerce platform manages product storage, searching, and recommendation systems. Construct a concept map linking Linear Data Structures → Sequential Access → Limitations → Non-Linear Data Structures → Hierarchical/Network Relationships → Applications. Analyze the differences between linear and non-linear structures and demonstrate how each supports specific components of the system. Justify which structure is more efficient for handling complex applications. (20 Marks)
4. A video streaming application must balance speed and memory usage. Construct a concept map connecting Time Complexity → Space Complexity → Tradeoff → Algorithm Design → Performance Optimization. Analyze how improving one factor affects the other and justify the optimal balance. (20 Marks)
5. A digital archive stores millions of documents. Construct a concept map linking Searching → Linear Search → Binary Search → Sorted Data → Time Complexity → Performance. Analyze the impact of data organization on search efficiency and justify the best technique.(20 Marks)

Rubrics for Calculation

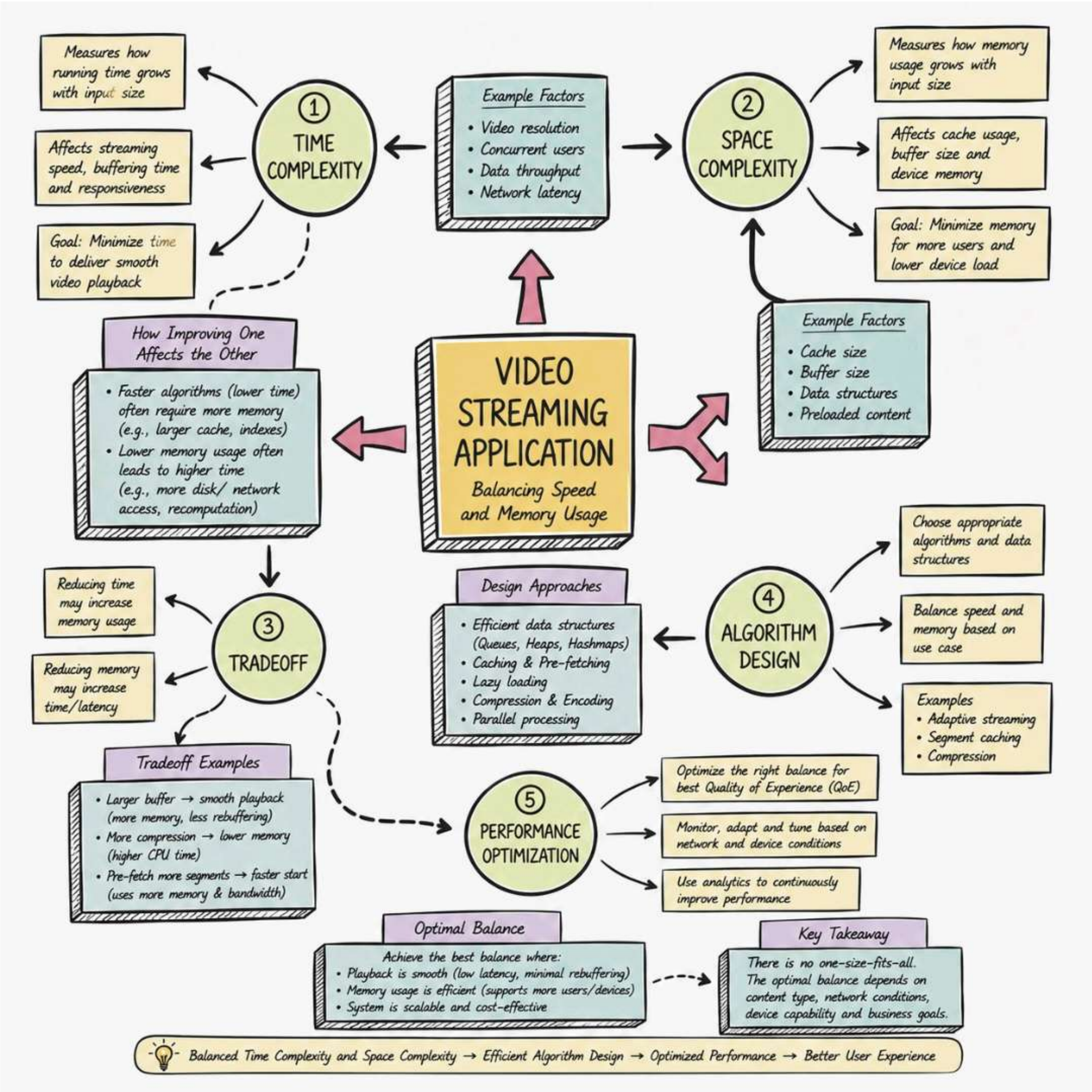
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand


Smart City Traffic Management System
(Manage vehicle movement, road networks, and signal control)
Primitive Data Structures (Basic Building Blocks)
Non-Primitive Data Structures (Derived from Primitive)
Linear Data Structures (Elements in Sequential Order)
Non-Linear Data Structures (Elements in Hierarchical or Network Order)
Examples
· Integer
· Float / Double
· Char
· Boolean
Examples
· Arrays
· Linked Lists
· Stacks
· Queues
· Trees
· Graphs
Arrays (Fixed Size, Same Type)
Linked Lists (Dynamic Size, Sequential Links)
Trees (Hierarchical Structure)
Graphs (Network of Nodes and Edges)
Applications in Traffic Management
· Store vehicle count
· Signal timings
· Speed limits
· Status flags (Red/Green)
Applications in Traffic Management
· Store & process large data
· Manage dynamic traffic flow
· Route optimization
· Signal control & scheduling
Applications
· Store traffic density of roads
· Maintain vehicle count in time slots
· Lookup signal timing table
Applications
· Manage list of vehicles in a lane
· Dynamic update of traffic events
· Incident tracking
Applications
· Represent road hierarchy (city > zone > road > lane)
· Decision trees for traffic signal control
· Priority routing
Applications
· Road networks as graphs (intersections as nodes, roads as edges)
· Shortest path (route optimization)
· Traffic flow analysis
Relationship & Justification
· Primitive structures store single values → foundation for all data handling.
· Non-Primitive structures organize and manage collections of data.
· Linear structures handle data in sequence → efficient for ordered data operations.
· Non-Linear structures model complex relationships → efficient for network & hierarchical data.
· Choosing the right structure improves performance, scalability, and real-time decision making in traffic management.


SMART HEALTHCARE SYSTEM CONCEPT MAP
SMART HEALTHCARE SYSTEM
(Efficient data organization, processing and improved patient care)
is the foundation
1. DATA
Raw facts collected from patients and devices
Examples: Temperature readings, Lab values, Patient ID
becomes
2. DATA OBJECT
Meaningful unit representing a specific piece of information
Examples: Patient Record, Medical History, Diagnosis
organized into
3. DATA STRUCTURE
Collection of related data objects
Examples: Array, Linked List, Structures (Records)
abstracted as
4. ABSTRACT DATA TYPE (ADT)
Logical view of data and the operations performed on it
Examples: ADT for Patient, Appointment, Medical Record
implemented as
5. IMPLEMENTATION IN C
Structures (struct)
Pointers (*)
Functions (Operations)
Efficient storage, processing and better system performance



←	LINEAR STRUCTURES	NON-LINEAR STRUCTURES
Structure	Sequential arrangement	Hierarchical / Network
Access	One by one	Multiple paths
Search	Slower (O(n))	Faster (O(log n) / O(1) avg)
Relationships	One-to-One / One-to-Many	Many-to-Many / Complex
Use Case	Simple data, small scale	Complex data, large scale
Scalability	Limited	High



VIDEO STREAMING APPLICATION
Balancing Speed and Memory Usage
Example Factors
• Video resolution
• Concurrent users
• Data throughput
• Network latency
1 TIME COMPLEXITY
Measures how running time grows with input size
Affects streaming speed, buffering time and responsiveness
Goal: Minimize time to deliver smooth video playback
2 SPACE COMPLEXITY
Measures how memory usage grows with input size
Affects cache usage, buffer size and device memory
Goal: Minimize memory for more users and lower device load
Example Factors
• Cache size
• Buffer size
• Data structures
• Preloaded content
How Improving One Affects the Other
• Faster algorithms (lower time) often require more memory (e.g., larger cache, indexes)
• Lower memory usage often leads to higher time (e.g., more disk/ network access, recomputation)
3 TRADEOFF
Reducing time may increase memory usage
Reducing memory may increase time/latency
Tradeoff Examples
• Larger buffer → smooth playback (more memory, less rebuffering)
• More compression → lower memory (higher CPU time)
• Pre-fetch more segments → faster start (uses more memory & bandwidth)
Design Approaches
• Efficient data structures (Queues, Heaps, Hashmaps)
• Caching & Pre-fetching
• Lazy loading
• Compression & Encoding
• Parallel processing
4 ALGORITHM DESIGN
Choose appropriate algorithms and data structures
Balance speed and memory based on use case
Examples
• Adaptive streaming
• Segment caching
• Compression
5 PERFORMANCE OPTIMIZATION
Optimize the right balance for best Quality of Experience (QoE)
Monitor, adapt and tune based on network and device conditions
Use analytics to continuously improve performance
Optimal Balance
Achieve the best balance where:
• Playback is smooth (low latency, minimal rebuffering)
• Memory usage is efficient (supports more users/devices)
• System is scalable and cost-effective
Key Takeaway
There is no one-size-fits-all. The optimal balance depends on content type, network conditions, device capability and business goals.
Balanced Time Complexity and Space Complexity → Efficient Algorithm Design → Optimized Performance → Better User Experience

SEARCHING IN DIGITAL ARCHIVE

(Millions of Documents)

1. SEARCHING

Finding a particular document from millions of records stored in a digital archive.

Goal: Fast, accurate and scalable search.

2. LINEAR SEARCH

Check each document one by one from the beginning until the desired document is found.

- Works on unsorted or sorted data
- Simple to implement

Not efficient for large datasets

3. BINARY SEARCH

Compare the middle element, if not match search in left or right half. Repeat until found.

- Requires sorted data
- Much faster than linear search

Ideal for large, sorted datasets

4. SORTED DATA

Binary search works only on sorted data.

- Data must be organized in ascending/descending order
- Sorting has an initial cost but enables fast searches later.

5. TIME COMPLEXITY

Technique	Best Case	Average Case	Worst Case
Linear Search	O(1)	O(n)	O(n)
Binary Search	O(1)	O(log n)	O(log n)

n = Number of documents in archive

6. PERFORMANCE

✓ Linear Search → Slow for large archives (time grows linearly with n)

✓ Binary Search → Fast and scalable (time grows logarithmically)

✓ Sorted data + Binary Search → Best performance for very large archives

CONCLUSION

Proper data organization (sorting) has a huge impact on search efficiency. For a digital archive with millions of documents, Binary Search on sorted data is the best technique due to lower time complexity and better scalability.